1. Database Objects ⭐ ⭐ ⭐ ⭐

Database Object matlab database ke andar stored koi bhi object.

Types of Database Objects

Object	Purpose
Database	Data store karna
Table	Rows aur Columns me data
View	Virtual Table
Index	Fast Searching
Stored Procedure	Reusable SQL Program
Function	Value Return karta hai
Trigger	Automatically execute hota hai
Sequence	Auto Number Generate

1. Views ⭐ ⭐ ⭐ ⭐

Definition

View ek **Virtual Table** hoti hai.

Isme data physically store nahi hota.

Ye sirf SELECT query ka result save karti hai.

```
Student Table
      |
      ▼
SELECT Query
      |
      ▼
VIEW
```

Create View

SQL

```sql
CREATE VIEW student_view AS
SELECT id,name,marks
FROM student;
```

View Dekho

SQL

```sql
SELECT *
FROM student_view;
```

Update View

```sql
CREATE OR REPLACE VIEW student_view AS
SELECT id,name,marks,city
FROM student;
```

Delete View

```sql
DROP VIEW student_view;
```

Index Kya Hai?

Index ek special data structure hai jo table me data ko jaldi search karne ke liye use hota hai.

Jaise kisi **book ke last me index** hota hai.

Agar tumhe "Database" topic dekhna hai to poori book nahi padhte.

Seedha index me page number dekhte ho.

Database me bhi wahi hota hai.

Purpose of Index

- ✅ Fast Searching (`WHERE`)
- ✅ Fast `JOIN`
- ✅ Fast `ORDER BY`
- ✅ Fast `GROUP BY`
- ✅ Fast `MIN()` / `MAX()`
- ✅ Performance Improve

Create Index

SQL

```sql
CREATE INDEX idx_name
ON student(name);
```

Composite Index

SQL

```sql
CREATE INDEX idx_city_marks
ON student(city,marks);
```

Show Index

SQL

```sql
SHOW INDEX
FROM student;
```

Drop Index

SQL

```sql
DROP INDEX idx_name
ON student;
```

Interview

Phone Search

SQL

```sql
CREATE INDEX idx_phone
ON student(phone);
```

Composite Example

SQL

```sql
CREATE INDEX idx_city_dept
ON student(city,dept_id);
```

Unique Index

Duplicate allow nahi karta.

SQL

```sql
CREATE UNIQUE INDEX idx_phone
ON student(phone);
```

Show Index

SQL

```sql
SHOW INDEX
FROM student;
```

Foreign Key Index

Foreign key par index **automatically har database me nahi banta**

Agar JOIN bahut hote hain, to manually banana achha hota hai.

SQL

```sql
CREATE INDEX idx_dept
ON student(dept_id);
```

Types of Index

Type	Purpose
Primary Index	Primary Key
Unique Index	Duplicate not allowed
Composite Index	Multiple Columns
Clustered Index*	Data sorted physically
Non-Clustered Index*	Separate index structure
Full Text Index	Text Search

2. Stored Procedure ⭐ ⭐ ⭐ ⭐ ⭐

Definition

Stored Procedure ek **precompiled SQL program** hota hai jo database me save rehta hai.

Ek baar likho, baar-baar call karo.

```
Application

↓

CALL Procedure

↓

Procedure

↓

Database
```

Create Procedure

SQL

```sql
DELIMITER $$

CREATE PROCEDURE GetStudents()
BEGIN
    SELECT * FROM student;
END $$

DELIMITER ;
```

Execute

SQL

```sql
CALL GetStudents();
```

Procedure with Parameter

SQL

```sql
DELIMITER $$

CREATE PROCEDURE StudentByCity
(
IN city_name VARCHAR(30)
)
BEGIN

SELECT *
FROM student
WHERE city=city_name;

END $$

DELIMITER ;
```

Execute

```sql
CALL StudentByCity('Indore');
```

Multiple Parameters

```sql
DELIMITER $$

CREATE PROCEDURE MarksGreater
(
IN m INT,
IN c VARCHAR(30)
)
BEGIN
```

Multiple Parameters

SQL

```sql
DELIMITER $$

CREATE PROCEDURE MarksGreater
(
IN m INT,
IN c VARCHAR(30)
)
BEGIN

SELECT *
FROM student
WHERE marks>m
AND city=c;

END $$

DELIMITER ;
```

Call

SQL

```sql
CALL MarksGreater(80,'Indore');
```

Drop Procedure

SQL

```sql
DROP PROCEDURE GetStudents;
```

Trigger Kya Hai?

Trigger ek database object hai jo kisi table par `INSERT`, `UPDATE`, ya `DELETE` hone par automatically execute hota hai.

Matlab trigger ko manually call nahi karna padta.

```
INSERT / UPDATE / DELETE
        ↓
     Trigger Fire
        ↓
Automatic Action
```

Types of Trigger

Trigger	Kab Chalta Hai
BEFORE INSERT	Insert se pehle
AFTER INSERT	Insert ke baad
BEFORE UPDATE	Update se pehle
AFTER UPDATE	Update ke baad
BEFORE DELETE	Delete se pehle
AFTER DELETE	Delete ke baad

```sql
DELIMITER $$

CREATE TRIGGER trigger_name

BEFORE/AFTER INSERT/UPDATE/DELETE

ON table_name

FOR EACH ROW

BEGIN

   SQL Statements;

END $$

DELIMITER ;
```

Log Table

SQL

```sql
CREATE TABLE student_log(

log_id INT AUTO_INCREMENT PRIMARY KEY,

student_id INT,

action VARCHAR(30),

action_time DATETIME

);
```

1. AFTER INSERT Trigger ⭐⭐

Student insert hote hi log table me entry ho.

SQL

```
DELIMITER $$

CREATE TRIGGER trg_after_insert

AFTER INSERT

ON student

FOR EACH ROW

BEGIN

INSERT INTO student_log

(student_id,action,action_time)

VALUES
```

```sql
VALUES

(NEW.id,'Inserted',NOW());

END $$

DELIMITER ;
```

Test

```sql
INSERT INTO student

VALUES

(66,'Raj',20,'Male','Indore',2,80,'9000000066',101);
```

2. AFTER DELETE Trigger ⭐

Delete hone par log save karo.

SQL

```
DELIMITER $$

CREATE TRIGGER trg_after_delete

AFTER DELETE

ON student

FOR EACH ROW

BEGIN

INSERT INTO student_log

(student_id,action,action_time)

VALUES
```

```
(student_id,action,action_time)

VALUES

(OLD.id,'Deleted',NOW());

END $$

DELIMITER ;
```

Test

```
SQL

DELETE

FROM student

WHERE id=66;
```

512cab-dcd0-83ee-a37b-da0623808l1a

3. BEFORE UPDATE Trigger ⭐

Negative marks allow nahi karna.

SQL

```
DELIMITER $$

CREATE TRIGGER trg_before_update

BEFORE UPDATE

ON student

FOR EACH ROW

BEGIN

IF NEW.marks<0 THEN

SET NEW.marks=0;

END IF;
```

```
SQL
SET NEW.marks=0;

END IF;

END $$

DELIMITER ;
```

Test

```
SQL
UPDATE student

SET marks=-50

WHERE id=2;
```

4. BEFORE INSERT Trigger

Marks 100 se jyada na ho.

SQL

```sql
DELIMITER $$

CREATE TRIGGER trg_before_insert

BEFORE INSERT

ON student

FOR EACH ROW

BEGIN

IF NEW.marks>100 THEN

SET NEW.marks=100;

END IF;
```

5. AFTER UPDATE Trigger

Marks update hone par log table me save karo.

SQL

```
DELIMITER $$

CREATE TRIGGER trg_after_update

AFTER UPDATE

ON student

FOR EACH ROW

BEGIN

INSERT INTO student_log

(student_id,action,action_time)

VALUES
```

```
VALUES

(NEW.id,'Marks Updated',NOW());

END $$

DELIMITER ;
```

OLD vs NEW ⭐ ⭐ ⭐ ⭐ ⭐

Keyword	Meaning
OLD	Update/Delete se pehle ka data
NEW	Insert/Update ke baad ka data

```
SHOW TRIGGERS;
```

Specific Trigger

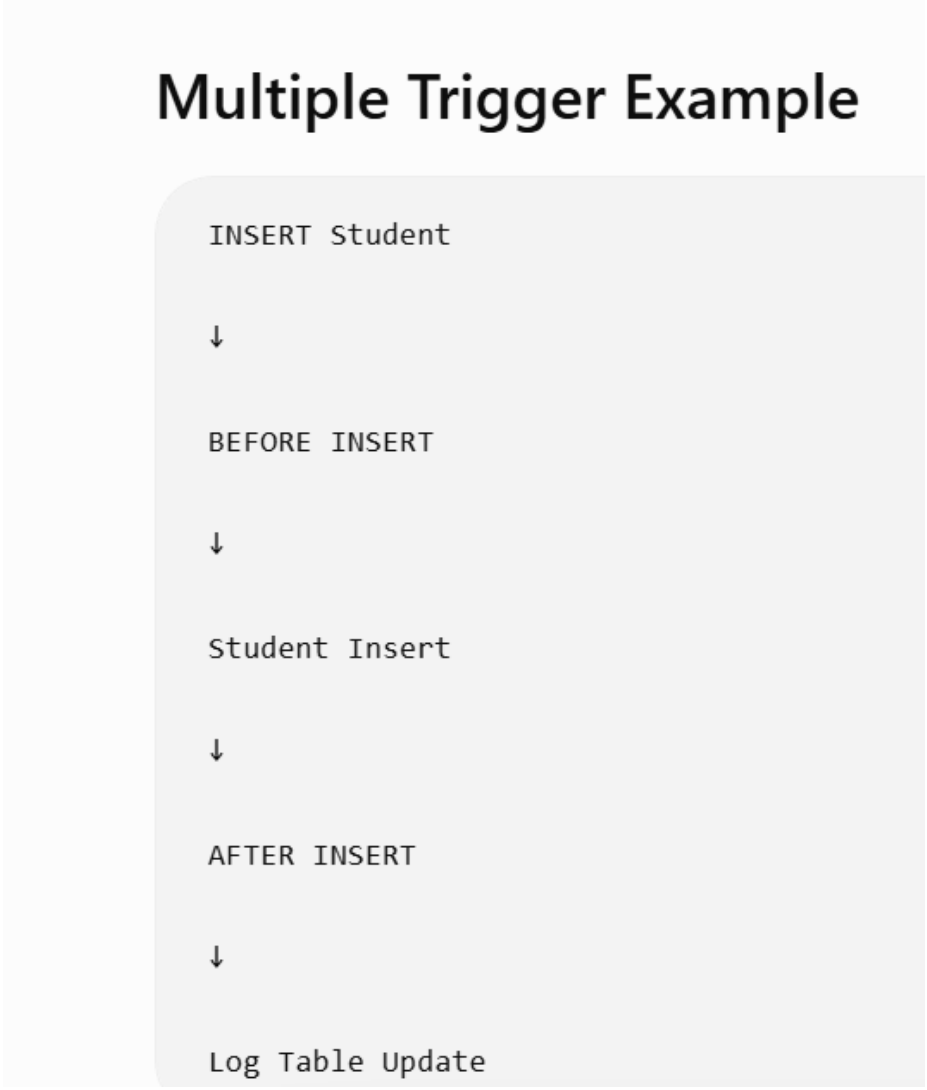
‹/› SQL

```
SHOW CREATE TRIGGER trg_after_insert;
```

Trigger Delete

‹/› SQL

```
DROP TRIGGER trg_after_insert;
```


Multiple Trigger Example
INSERT Student
↓
BEFORE INSERT
↓
Student Insert
↓
AFTER INSERT
↓
Log Table Update

1. Salary Negative Nahi Honi Chahiye

SQL

```sql
IF NEW.salary<0 THEN

SET NEW.salary=0;

END IF;
```

2. Mobile Number 10 Digit

SQL

```sql
IF LENGTH(NEW.phone)!=10 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT='Invalid Phone Number';

END IF;
```

3. Marks Greater Than 100

SQL

```sql
IF NEW.marks>100 THEN

SET NEW.marks=100;

END IF;
```

---

4. Prevent Delete

SQL

```sql
SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT='Delete Not Allowed';
```

Trigger	Stored Procedure
Automatically chalta hai	Manually `CALL` karna padta hai
Event par execute hota hai	Jab user call kare tab
INSERT/UPDATE/DELETE se trigger hota hai	Kisi bhi time execute ho sakta hai

Trigger vs Function

Trigger	Function
Auto execute	`SELECT` se call hoti hai
Event based	Value return karti hai
Return value nahi hoti	Return value mandatory hoti hai

Function Kya Hai?

Function SQL ka ek database object hai jo input (parameters) leta hai, kuch processing karta hai aur hamesha ek value return karta hai.

> **Rule:** SQL Function ka **return value mandatory** hota hai.

Real Life Example

Calculator

```
Number = 25

↓

SQRT()

↓

5
```

Purpose of Function

- ✅ Calculation karna
- ✅ Reusable code likhna
- ✅ Business logic implement karna
- ✅ Data formatting
- ✅ Validation
- ✅ Code duplication kam karna

SQL

```sql
DELIMITER $$

CREATE FUNCTION function_name(parameter datatype)

RETURNS datatype

DETERMINISTIC

BEGIN

    SQL Statements

    RETURN value;

END $$

DELIMITER ;
```

Example 1 (Bonus Function)

SQL

```sql
DELIMITER $$

CREATE FUNCTION Bonus(m INT)

RETURNS INT

DETERMINISTIC

BEGIN

RETURN m+5;

END $$

DELIMITER ;
```

.com/c/6a512cab-dcd0-83ee-a37b-da062380811a

Example 2 (Pass/Fail Function)

SQL

```sql
DELIMITER $$

CREATE FUNCTION Result(m INT)

RETURNS VARCHAR(10)

DETERMINISTIC

BEGIN

IF m>=40 THEN

RETURN 'PASS';

ELSE

RETURN 'FAIL';

END IF;
```

Table ke saath

SQL

```sql
SELECT

name,

marks,

Result(marks)

FROM student;
```

/6a512cab-dcd0-83ee-a37b-da062380811a

Example 3 (Grade Function)

SQL

```sql
DELIMITER $$

CREATE FUNCTION Grade(m INT)

RETURNS CHAR(1)

DETERMINISTIC

BEGIN

IF m>=90 THEN

RETURN 'A';

ELSEIF m>=80 THEN

RETURN 'B';
```

```
RETURN 'B';

ELSEIF m>=70 THEN

RETURN 'C';

ELSE

RETURN 'D';

END IF;

END $$

DELIMITER ;
```

Use

```
SQL

SELECT

name,

marks,

Grade(marks)

FROM student;
```

Example 4 (Age Category)

SQL

```sql
DELIMITER $$

CREATE FUNCTION AgeCategory(a INT)

RETURNS VARCHAR(20)

DETERMINISTIC

BEGIN

IF a<18 THEN

RETURN 'Minor';

ELSE

RETURN 'Adult';

END IF;
```

Show Functions

SQL

```sql
SHOW FUNCTION STATUS;
```

Delete Function

SQL

```sql
DROP FUNCTION Bonus;
```

Function vs Stored Procedure

Function	Stored Procedure
Value return karna mandatory	Return optional
`SELECT` se call hoti hai	`CALL` se execute hoti hai
Calculation ke liye	Business operations ke liye
DML (INSERT/UPDATE/DELETE) ka use generally nahi karte	DML operations kar sakti hai

Function vs Trigger

Function	Trigger
User call karta hai	Automatically execute hota hai
Value return karti hai	Return value nahi hoti
`SELECT` me use hoti hai	Event (`INSERT/UPDATE/DELETE`) par chalti hai

Window Function Kya Hai?

Window Function rows par calculation karti hai bina rows ko collapse kiye.

Normal Aggregate Function

```sql
SELECT dept_id,AVG(marks)
FROM student
GROUP BY dept_id;
```

Window Function

```sql
SELECT
name,
marks,
AVG(marks) OVER()
FROM student;
```

Purpose

- ✅ Ranking
- ✅ Running Total
- ✅ Department Wise Topper
- ✅ Previous Row Compare
- ✅ Next Row Compare
- ✅ Moving Average
- ✅ Salary Comparison
- ✅ Percentage Calculation

Syntax

```sql
Function()

OVER(

PARTITION BY column

ORDER BY column

)
```

OVER()

Ye batata hai calculation kis rows ke group (window) par hogi.

SQL

```sql
SELECT
name,
marks,
AVG(marks)
OVER()
FROM student;
```

PARTITION BY

Rows ko groups me divide karta hai.

```
Student

↓

Department Wise Groups

↓

Calculation
```

Example

Example

SQL

```sql
SELECT
name,
dept_id,
marks,
AVG(marks)
OVER(PARTITION BY dept_id)
FROM student;
```

ORDER BY

Ranking ya sequence ke liye.

SQL

```sql
SELECT
name,
marks,
ROW_NUMBER()
OVER(ORDER BY marks DESC)
FROM student;
```

Types of Window Functions

Function	Purpose
ROW_NUMBER()	Unique Rank
RANK()	Same Rank, Gap
DENSE_RANK()	Same Rank, No Gap
NTILE()	Groups me divide
LEAD()	Next Row
LAG()	Previous Row
FIRST_VALUE()	First Value
LAST_VALUE()	Last Value

jpt.com/c/6a512cab-dcd0-83ee-a37b-da06238081fa

LAG()	Previous Row
FIRST_VALUE()	First Value
LAST_VALUE()	Last Value
SUM() OVER()	Running Total
AVG() OVER()	Running Average
COUNT() OVER()	Running Count
MAX() OVER()	Maximum
MIN() OVER()	Minimum

1 ROW_NUMBER()

Har row ko unique number deta hai.

SQL

```sql
SELECT
name,
marks,
ROW_NUMBER()
OVER(ORDER BY marks DESC) AS rn
FROM student;
```

Department Wise

SQL

```sql
SELECT
name,
dept_id,
marks,
ROW_NUMBER()
OVER(
PARTITION BY dept_id
ORDER BY marks DESC
) rn
FROM student;
```

Window Function vs GROUP BY

GROUP BY	Window Function
Rows combine ho jaati hain	Rows same rehti hain
Aggregate result deta hai	Aggregate + original rows dono deta hai
Detail rows nahi milti	Detail rows milti hain
Ranking possible nahi	Ranking possible hai

RANK() Function in SQL ⭐ ⭐ ⭐ ⭐ ⭐ (Window Function)

RANK() Kya Hai?

RANK() ek Window Function hai jo rows ko rank (position) deta hai. Agar do ya zyada rows ki value same ho, to unhe same rank milti hai aur next rank skip ho jati hai.

Example:

Marks

```
97
97
95
90
```

Rank

```
97 → Rank 1
```

n/c/6a512cab-dcd0-83ee-a37b-da062380811a

Syntax

SQL

```sql
SELECT
column1,
column2,
RANK() OVER(
ORDER BY column DESC
) AS rank
FROM table_name;
```

Example 1 (Overall Ranking)

SQL

```sql
SELECT
id,
name,
marks,
RANK() OVER(
ORDER BY marks DESC
) AS student_rank
FROM student;
```

Example 3 (Department Wise Ranking)

Har department ke andar ranking.

SQL

```sql
SELECT
name,
dept_id,
marks,
RANK() OVER(
PARTITION BY dept_id
ORDER BY marks DESC
) AS dept_rank
FROM student;
```

Example 4 (Topper of Each Department)

SQL

```sql
SELECT *
FROM
(
    SELECT *,
           RANK() OVER(
               PARTITION BY dept_id
               ORDER BY marks DESC
           ) AS rnk
    FROM student
) t
WHERE rnk = 1;
```

Example 5 (Top 3 Students of Every Department)

SQL

```sql
SELECT *
FROM
(
    SELECT *,
           RANK() OVER(
               PARTITION BY dept_id
               ORDER BY marks DESC
           ) AS rnk
    FROM student
) t
WHERE rnk <= 3;
```

Example 6 (Second Highest Marks)

SQL

```sql
SELECT *
FROM
(
    SELECT *,
           RANK() OVER(
               ORDER BY marks DESC
           ) AS rnk
    FROM student
) t
WHERE rnk = 2;
```

> **Note:** Agar highest marks tie hain (do students ke 97 marks), to `RANK()` me next rank **3** hogi. Isliye second highest unique marks nikalne ke liye aksar `DENSE_RANK()` zyada suitable hota hai.

MAX() with GROUP BY

Har department ka highest marks.

SQL

```sql
SELECT
dept_id,
MAX(marks) AS highest_marks
FROM student
GROUP BY dept_id;
```

MAX() with HAVING

Sirf un departments ko dikhao jinke highest marks 90 se jyada hain.

SQL

```sql
SELECT
dept_id,
MAX(marks) AS highest_marks
FROM student
GROUP BY dept_id
HAVING MAX(marks) > 90;
```

MAX() with JOIN

Department Name ke saath Highest Marks.

SQL

```sql
SELECT
d.dept_name,
MAX(s.marks) AS highest_marks
FROM student s
JOIN department d
ON s.dept_id = d.dept_id
GROUP BY d.dept_name;
```

MAX() with Subquery

Highest marks wale student ki details.

SQL

```sql
SELECT *
FROM student
WHERE marks = (
    SELECT MAX(marks)
    FROM student
);
```

Second Highest Marks

SQL

```sql
SELECT MAX(marks)
FROM student
WHERE marks < (
    SELECT MAX(marks)
    FROM student
);
```

Third Highest Marks

SQL

```sql
SELECT MAX(marks)
FROM student
WHERE marks < (
    SELECT MAX(marks)
    FROM student
    WHERE marks < (
        SELECT MAX(marks)
        FROM student
    )
);
```

.com/c/6a512cab-dcd0-83ee-a37b-da062380811a

Highest Marks in Each City

SQL

```sql
SELECT
city,
MAX(marks) AS highest_marks
FROM student
GROUP BY city;
```

Highest Marks in Each Department (with Student Name)

SQL

```sql
SELECT s.*
FROM student s
WHERE marks = (
    SELECT MAX(marks)
    FROM student
    WHERE dept_id = s.dept_id
);
```

MAX() as Window Function

Har student ke saath uske department ka maximum marks.

SQL

```sql
SELECT
name,
dept_id,
marks,
MAX(marks) OVER(PARTITION BY dept_id) AS dept_max
FROM student;
```

CTE Kya Hai?

CTE (Common Table Expression) ek temporary result set hota hai jo query ke execution ke dauran hi exist karta hai.

- Ye permanent table **nahi** hota.
- Sirf us query ke liye available hota hai.
- Query khatam → CTE bhi khatam.

Socho ki CTE ek **temporary virtual table** hai.

Purpose of CTE

- ✅ Complex query ko simple banana
- ✅ Readability improve karna
- ✅ Subquery ki jagah use karna
- ✅ Recursive queries likhna
- ✅ Multiple baar same result use karna
- ✅ Ranking aur Window Functions ke saath use karna

Example 1 – Basic CTE

80 se jyada marks wale students.

SQL

```sql
WITH topper AS
(
    SELECT *
    FROM student
    WHERE marks > 80
)

SELECT *
FROM topper;
```

Example 2 – Aggregate Function

Average marks nikalna.

SQL

```sql
WITH avg_marks AS
(
    SELECT AVG(marks) AS avg_mark
    FROM student
)

SELECT *
FROM avg_marks;
```

Example 3 – Students Above Average

SQL

```sql
WITH avg_table AS
(
    SELECT AVG(marks) AS avg_marks
    FROM student
)

SELECT *
FROM student
WHERE marks >
(
    SELECT avg_marks
    FROM avg_table
);
```

Example 4 – Department Wise Average

SQL

```sql
WITH dept_avg AS
(
    SELECT
        dept_id,
        AVG(marks) AS average_marks
    FROM student
    GROUP BY dept_id
)

SELECT *
FROM dept_avg;
```

Example 5 – JOIN with CTE

SQL

```sql
WITH dept_avg AS
(
    SELECT
        dept_id,
        AVG(marks) avg_marks
    FROM student
    GROUP BY dept_id
)

SELECT
d.dept_name,
a.avg_marks
FROM dept_avg a
JOIN department d
ON a.dept_id=d.dept_id;
```

Example 6 – Multiple CTE

SQL

```sql
WITH avg_marks AS
(
    SELECT AVG(marks) avg_mark
    FROM student
),

max_marks AS
(
    SELECT MAX(marks) max_mark
    FROM student
)

SELECT
avg_mark,
max_mark
FROM avg_marks,max_marks;
```

Example 7 – Window Function + CTE

Department Wise Rank

SQL

```sql
WITH ranking AS
(
SELECT

name,

dept_id,

marks,

RANK() OVER(

PARTITION BY dept_id

ORDER BY marks DESC

) rnk
```

Example 8 – Highest Marks Student

SQL

```sql
WITH highest AS
(
SELECT MAX(marks) highest_marks

FROM student
)

SELECT *

FROM student

WHERE marks=

(

SELECT highest_marks

FROM highest

);
```

Example 9 – Lowest Marks

SQL

```sql
WITH lowest AS
(
SELECT MIN(marks) m

FROM student
)

SELECT *

FROM student

WHERE marks=

(

SELECT m

FROM lowest

);
```

Recursive CTE Kya Hai?

Recursive CTE ek special type ka CTE hai jo khud ko (itself) baar-baar call karta hai jab tak condition false na ho jaye.

Ye recursion (loop) ki tarah kaam karta hai.

Use Cases:

- Employee Hierarchy (CEO → Manager → Employee)
- Folder Structure
- Category Tree
- Family Tree
- 1 se N tak numbers generate karna
- Graph Traversal

SQL

```sql
WITH RECURSIVE cte_name AS
(
    -- Anchor Query (Starting Point)

    SELECT ...

    UNION ALL

    -- Recursive Query

    SELECT ...
    FROM cte_name
    WHERE condition
)

SELECT *
FROM cte_name;
```

Recursive CTE ke **2 parts** hote hain:

1. **Anchor Query** → Starting value
2. **Recursive Query** → Repeat hoti hai

Recursive CTE vs Normal CTE

Normal CTE	Recursive CTE
Sirf ek baar execute hota hai	Baar-baar execute hota hai
Self reference nahi hota	Khud ko reference karta hai
Complex query simplify karta hai	Hierarchy aur sequence generate karta hai
`WITH`	`WITH RECURSIVE`

Example 1: 1 se 10 Print Karo

SQL

```sql
WITH RECURSIVE numbers AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n + 1
    FROM numbers
    WHERE n < 10
)

SELECT *
FROM numbers;
```

Example 3: Even Numbers

SQL

```sql
WITH RECURSIVE even_no AS
(
    SELECT 2 AS n

    UNION ALL

    SELECT n+2
    FROM even_no
    WHERE n<20
)

SELECT *
FROM even_no;
```

Example 4: Odd Numbers

SQL

```sql
WITH RECURSIVE odd_no AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n+2
    FROM odd_no
    WHERE n<19
)

SELECT *
FROM odd_no;
```

Example 5: Multiplication Table of 5

SQL

```sql
WITH RECURSIVE table5 AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n+1
    FROM table5
    WHERE n<10
)

SELECT
n,
5*n AS result
FROM table5;
```

Example 6: Factorial

SQL

```sql
WITH RECURSIVE fact AS
(
    SELECT
        1 AS n,
        1 AS factorial

    UNION ALL

    SELECT
        n+1,
        factorial*(n+1)
    FROM fact
    WHERE n<5
)

SELECT *
FROM fact;
```

```sql
WITH RECURSIVE hierarchy AS
(
    SELECT
        emp_id,
        emp_name,
        manager_id
    FROM employee
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id
    FROM employee e
    JOIN hierarchy h
      ON e.manager_id = h.emp_id
)

SELECT *
FROM hierarchy;
```

Example 8: Countdown

SQL

```sql
WITH RECURSIVE countdown AS
(
    SELECT 10 AS n

    UNION ALL

    SELECT n-1
    FROM countdown
    WHERE n>1
)

SELECT *
FROM countdown;
```

Output

Example 9: Sum 1 to 10

SQL

```sql
WITH RECURSIVE numbers AS
(
    SELECT 1 AS n

    UNION ALL

    SELECT n+1
    FROM numbers
    WHERE n<10
)

SELECT SUM(n)
FROM numbers;
```